VAYU ORBIT

Target System Specification

Version 1.1 | Revised after specification review | Owner approval draft

DESIGN BASELINE — NOT IMPLEMENTATION AUTHORIZATION

Document status

This is a revised design baseline following specification review. It is an owner-approval draft, not implementation authorization. Do not change code, database, schema, migrations, deployment configuration, or production data until the owner explicitly approves a defined implementation phase.

Labels: CONFIRMED CONTEXT = previously expressed requirement; RECOMMENDED BASELINE = default recommended after the owner asked which option is better, still pending explicit approval; OPEN = requires owner input or evidence.

1. Purpose and constraints

VAYU ORBIT is a convoy-movement planning and logistics decision-support system aligned with Army Design Bureau PDS-3: “Intelligent Convoy Movement Planning and Dynamic Route Optimization System for Military Logistics Decision Support.”

- Support journey creation, route planning/review/clearance, vehicle and personnel readiness, checkpoint inspection, controlled start, monitoring, disruption handling, and reporting.
- Do not invent doctrine, tactical formations, vehicle counts, consumption formulas, route clearance, or readiness facts.
- Unknown or stale data must never silently become Ready, Clear, Safe, Approved, or Pass.
- This specification does not authorize implementation.

2. Current-system baseline and evidence limits

Prior read-only audit evidence described FastAPI/SQLite, React/Vite, NetworkX-based route logic, and current technical roles CREATOR, CHECKPOINT_OPERATOR, COORDINATOR, and ADMIN. The target six-role model therefore needs explicit mapping and migration design.

The audit reported that all logged-in roles could see 13 sidebar pages; journey status changes used a broad operational-role check that included checkpoint operators; route-change requests, Company Commander decisions, ROP clearance records, and a formal checkpoint-inspection model were not found in the audited scope. The diagnostic database endpoint was reported unauthenticated. These are prior audit findings, not a claim that the repository was re-audited for this revision.

Offline operation is not proven. Prior audit evidence identified online map tiles, a TomTom SDK CDN, and Google Fonts CDN. Verify any TomTom routing/API usage directly in current source. Do not claim offline readiness until network-isolation tests pass.

Next safe step: read-only validation against the current repository/schema, with file/line evidence for roles, state enums, route representation, endpoint authorization, external dependencies, and legacy data. Do not modify production files or data.

3. Six-role target model

Target roles: Journey Creator; ROP Operator; Company Commander; Checkpoint Operator; Journey Coordinator; System Administrator. Software roles are distinct from vehicle crews and support-team records.

RECOMMENDED BASELINE: separate accounts per role, with controlled multi-role assignment only when justified. Creator/Coordinator overlap may be allowed only with recorded authorization. A route-change requester cannot approve their own request. ROP Operator and Company Commander should be separate people on a journey. These rules need owner approval.

System Administrator manages local configuration and user accounts but does not automatically bypass operational approval, clearance, inspection, fuel, or start gates.

Role	Responsibilities	Restrictions
Journey Creator	Create journey, configure manifest, save/submit, view authorized reports	Cannot self-approve route change or start unless separately authorized as Coordinator
ROP Operator	Review assigned route; request/record alternatives; clear exact route version	Clearance is not commander approval or start authorization
Company Commander	Approve/reject route-change request with reason	Does not perform ROP clearance; cannot approve own request
Checkpoint Operator	Inspect assigned vehicles/personnel; record pass/fail/exceptions	Cannot start/end journeys or approve/change routes
Journey Coordinator	Validate start gates; start, monitor, handle operational state, close journey	Cannot bypass failed gates without approved policy
System Administrator	Manage accounts/settings and controlled assignment corrections	No implicit operational override

4. Journey lifecycle and mandatory gates

Proposed lifecycle: DRAFT → SCHEDULED → ACTIVE → DISRUPTED (when applicable) → ACTIVE after controlled resume; ACTIVE/DISRUPTED → COMPLETED. Cancellation/abort permissions and transitions must be defined before implementation; confirm existing state names against current code.

RECOMMENDED BASELINE: start only when all mandatory gates pass together: current route version has valid ROP clearance; any required route-change approval exists; required checkpoint inspections pass; existing fuel-feasibility gate passes; journey/route version has not changed since validation; and the Coordinator is authorized. Validation and state transition must be atomic on the authoritative server.

Version the mandatory gate checklist. Each gate needs an owner, evidence, status values, freshness/expiry rule, blocking behavior, and audit event. Missing or stale evidence is not a pass. No overrides are recommended for initial release. Backend—not just UI—must enforce gates and transitions.

Blocked actions must state the failing gate and next authorized action.

5. Route alternatives, versioning, approval, clearance

Persist route alternatives as explicit versions, not temporary UI selections. A route version should contain journey ID, monotonic version number, ordered nodes/waypoints, canonical geometry/route hash, routing/data-source version where available, creator/time, and supersession relationship.

A route-change request records requester, current and proposed versions, reason, status, and decision metadata. Company Commander approves/rejects with a reason; ROP Operator separately clears the exact approved version. Approval and clearance are separate records and checks.

A material route change creates a new version and invalidates clearance for the superseded version. Start validation must reject stale approval/clearance and concurrent changes; version checks and start must be transactional.

RECOMMENDED BASELINE: initial route requires ROP clearance; Company Commander approval is required for requested alternatives, not every initial route. Active-journey rerouting needs a separately approved workflow with route versioning, approval, clearance, and auditability. No fast path until authority and criteria are defined.

6. Checkpoint inspection and coordination

Checkpoint inspection is a formal record separate from journey status. Record journey, checkpoint, assigned operator, inspected vehicle/personnel, checklist version, result, exceptions, timestamps, and audit history.

The authorized owner must supply the actual inspection checklist; do not invent doctrine. Mandatory items block passage/start if failed or incomplete. Recommended initial baseline: no checkpoint override.

Coordinator starts only after server-side gate validation; monitors state, alerts, checkpoint events and disruptions; then closes/completes under defined rules. Creator and Coordinator may access reports subject to assignment/authorization.

Distance, speed, spacing and freshness thresholds must be owner-approved. Alerts show source, timestamp, severity, and whether measured, manually entered, or estimated.

7. Convoy manifest, crew, readiness

Manifest is configurable per mission; do not hard-code formations/counts. It may include lead/commander vehicle, communications/rover, passenger buses, ration/supply trucks, fuel trucks, water tankers, jammer-equipment vehicles, QRT vehicles, ambulances/medical teams, and optional rear vehicles.

Illustration only: 400 personnel at 20 passengers per bus implies 20 buses. This is not doctrine or a default. Actual passenger count, rated capacity, and crew count are separate fields.

Track serviceability, journey assignment, crew readiness, equipment readiness, communications status and medical-team staffing separately. Each status needs source/time; unknown or stale remains unknown/stale. Crews/support personnel are records, not software roles.

OPEN: “three teams of three doctor teams” is ambiguous—three teams with three staff each, or three doctors total. Do not encode a count until clarified.

Jammer assets may be tracked for authorized inventory/readiness only. Do not add RF jamming, interference, or suppression controls.

8. Fuel feasibility and onboard fuel

Verify existing fuel-feasibility behavior in current source and preserve it during review. Previously approved rule: block departure by default when verified data shows the convoy cannot reach an authorized suitable refuelling point while preserving required reserve; unknown/stale fuel or station data warns and is not safe; any override would require authorized role, reason, and audit.

A later preference raised carrying an onboard fuel truck to reduce dependence on public stations. RECOMMENDED BASELINE: do not weaken/remove the existing gate yet. Treat onboard refuelling as an additional capability only after validating fuel-truck capacity, compatibility, allocation, transfer constraints, per-vehicle usable fuel/reserve and data provenance. Do not invent consumption rates.

OPEN: decide whether authorized military fuel points, onboard fuel trucks, or both count toward feasibility, and who verifies inputs. Until then, leave current behavior unchanged.

9. Offline deployment and local network

CONFIRMED CONTEXT: support (A) one isolated offline computer and (B) multiple authorized devices on an isolated LAN without internet/cloud dependency. A local server is authoritative; clients must not directly share/open the SQLite file.

RECOMMENDED BASELINE: evaluate licensed local raster/vector maps and a schematic route-only fallback. Decide only after checking coverage, licensing, storage, performance and data provenance. Online services/CDNs cannot be assumed to work offline; verify TomTom SDK/routing usage and license terms.

Offline mode must make no external requests, including fonts, tiles, telemetry, geocoding, analytics or update checks. Verify static dependencies and controlled network-isolation/outbound traffic. UI must distinguish full local map coverage from schematic/unavailable data.

For LAN mode, define local-server binding, device authorization, network boundaries, transport protection, authoritative timestamps, backup/restore, session handling, conflicts and reconnect behavior. RECOMMENDED BASELINE: writes require a live connection to the authoritative server; no silent queuing of conflicting operational writes.

Test SQLite concurrency, locking, backup recovery and failure behavior under expected workload before finalizing architecture.

10. Authorization, audit, integrity

- Enforce roles, journey assignments and object-level access in backend endpoints; hiding UI pages is not authorization.
- Protect diagnostics and API documentation under explicit policy; remove unauthenticated sensitive database diagnostics from operational use.
- Use secure password handling, configurable secrets, session expiry/revocation, unique accounts and audited account/role changes. Never ship demo credentials or hard-coded production secrets.
- Audit journey submission/assignment, route versions/requests/decisions/clearance, inspections, gate failures, start/resume/disruption/completion/cancellation, any approved overrides, and admin changes.
- Use transactions for state/gate checks. Test duplicates, stale clients, simultaneous route changes, concurrent starts and database contention.
- Back up authoritative data and required map/configuration files; test restoration. Test scripts use isolated databases unless explicitly authorized otherwise.

11. UI and reporting

- Role-appropriate navigation must match backend permissions.
- Show journey state, current route version, gate status, source/time, checkpoint progress and exceptions without presenting unknown data as ready.
- Explain blocked actions and identify the failing gate and next responsible role/action.
- Authorized Creator and Coordinator reports include journey identity, route version, decisions/clearance, inspections, state transitions, exceptions and timestamps.
- Offline UI shows server connection and map-data state; reconnect refreshes authoritative state before consequential actions.

12. Validation plan

Phase 0 — read-only repository validation: compare specification with current code/schema; cite file paths/lines; separate facts from assumptions; identify migration impacts. No code/database changes.

Phase 1 — design closure: settle owner decisions; approve route versions, state transitions, gate catalogue, role mapping, legacy policy, map option, data model and acceptance tests.

Phase 2 — isolated proof only after approval, using a separate test database. Do not alter production data.

Phase 3 — implementation only after separate milestone authorization. Each milestone includes changed files, migration/rollback plan, automated tests, manual acceptance evidence and limitations. Claude reviews final brief before Antigravity implementation.

13. Acceptance criteria

- Every backend endpoint enforces role, assignment and object-level access; negative tests prove prohibited actions fail.
- Route alternatives are persisted/versioned; commander decision and ROP clearance are separate and bound to the current version.
- Route-version change invalidates old clearance and cannot race through start.
- Start is denied unless all configured mandatory gates pass in one authoritative transaction.
- Checkpoint operators cannot start/end journeys or approve/change routes.
- Unknown/stale vehicle, crew, equipment, communications, fuel or checkpoint data never counts as a pass.
- Legacy journeys follow an approved policy without silent bypass or accidental loss of access.
- Single-host and isolated-LAN modes are tested; network capture shows no external requests.
- Concurrency, backup/restore, stale-client, unauthorized-access and state-transition tests pass.
- No production code/data changes before explicit approval of implementation milestone.

14. Owner decisions and open questions

ID	Recommended default / proposal	Status
D1 Route approval	ROP clearance for initial route; commander approval for requested alternatives	Approve/revise
D2 Role overlap	Separate accounts; recorded Creator/Coordinator exception; requester cannot approve own request; ROP/Commander separate per journey	Approve/revise
D3 Checklist	Owner-supplied versioned checklist; mandatory items block; no initial override	Checklist content open
D4 Fuel	Preserve existing gate; validate onboard fuel model before integration	Decide what counts as refuelling point
D5 Maps	Evaluate licensed local tiles/vector maps plus schematic fallback	Choose after evidence
D6 Deployment	Single offline host and isolated-LAN clients; writes only while connected	Validate technically
D7 Assignment	Creator assigns at submission; audited admin reassignment under defined conditions	Approve/revise
D8 Readiness	Manual status first; verified telemetry later; Unknown/Stale remains so	Freshness limits open
D9 Active reroute	New route version and renewed approval/clearance; no shortcut until defined	Design required
D10 Legacy journeys	Preserve and label legacy; controlled review before applying new gates	Transition policy open
D11 Medical staffing	Clarify three teams of three staff vs three doctors total	Owner clarification
D12 Alert thresholds	Owner-approved speed, spacing, checkpoint-distance and freshness thresholds	Owner inputs

Approval checkpoint: do not implement schema changes, migrations, role changes, gate enforcement, route workflow, map replacement, or production-data updates until the owner approves the relevant design and milestone. The next safe action is read-only design validation against the current repository.